From Bigrams to GPT

A minimal path to understanding and implementing transformer language models

Redes Neuronales 2026

Dr. Juan I. Perotti
FAMAF-UNC, IFEG-CONICET

2026

Abstract

These notes build a GPT-style language model from first principles, in the smallest number of conceptual steps that still ends at a real, trainable architecture. We start by recasting next-token prediction as the problem of embedding the context into a vector, fit the simplest such model (a bigram) first by counting and then by gradient descent, add attention as a mechanism for tokens to share information across positions, assemble attention and a feed-forward network into a transformer block, and stack blocks into a GPT. Each chapter has a directly corresponding implementation in `code/`, written in PyTorch using ordinary tensors, `nn.Linear`, `nn.Embedding` and `softmax` — not `nn.MultiheadAttention` or `nn.Transformer`, so that every operation the model performs is visible. We rely on PyTorch's automatic differentiation and optimizers (`loss.backward()`, `torch.optim.AdamW`) rather than reimplementing them: the object of study here is the *architecture*, not the training machinery.

Contents

1 Language modeling **3**
- 1.1 The task . . . 3
- 1.2 Measuring fit: negative log-likelihood and perplexity . . . 4

2 Embedding the context **6**
- 2.1 From contexts to vectors . . . 6
- 2.2 Logits and softmax . . . 7
- 2.3 The loss in terms of logits . . . 7
- 2.4 Token embeddings . . . 8

3 n-gram and bigram models **9**
- 3.1 Truncating the context: n-gram models . . . 9
- 3.2 Fitting an n-gram model by counting . . . 10
- 3.3 Limitations of counting . . . 10
- 3.4 The case $n = 2$: bigrams . . . 11
- 3.5 The same model, trained by gradient descent . . . 13
- 3.6 The real limitation, and what a fix would look like . . . 14

4 Self-attention **16**
- 4.1 Queries, keys and values . . . 16
- 4.2 Why the $\sqrt{d_k}$ . . . 16
- 4.3 Matrix form and the causal mask . . . 17
- 4.4 Computational cost . . . 17

5 The transformer block **19**
- 5.1 Multi-head attention . . . 19
- 5.2 LayerNorm . . . 20
- 5.3 The feed-forward network . . . 21
- 5.4 Residual connections and the full block . . . 22

6 Assembling GPT **24**
- 6.1 Why position has to be added by hand . . . 24
- 6.2 The full forward pass . . . 25
- 6.3 Weight tying . . . 26
- 6.4 Parameter count . . . 26

7 Training and generation **28**
7.1 Teacher forcing and batching 28
7.2 Optimization: what autograd and AdamW are doing 28
7.3 Generation 29
7.4 Where this is going 30

Chapter 1

Language modeling

1.1 The task

A *language model* assigns a probability to every possible sequence of tokens $x_1, x_2, \ldots, x_T$ drawn from a fixed vocabulary of size V. A token can be a character, a sub-word piece, or a word. We postpone the question of *how* text is turned into tokens to Remark 1.2 and work with characters until Chapter 7.

Modeling the joint distribution $p(x_1, \ldots, x_T)$ directly is intractable for any interesting T, because the number of sequences grows as V^T. The chain or Bayes rule of probability

$$p(x, y) = p(x|y)p(y),$$

turns this into something more tractable by factoring the joint into a product of conditionals,

$$\begin{aligned} p(x_1, \ldots, x_T) &= p(x_2, \ldots, x_T \mid x_1)p(x_1) \\ &= p(x_3, \ldots, x_T \mid x_2, x_1)p(x_2 \mid x_1)p(x_1) \\ &= p(x_4, \ldots, x_T \mid x_3, x_2, x_1)p(x_3 \mid x_2, x_1)p(x_2 \mid x_1)p(x_1) \\ &\ldots \\ &= \prod_{t=1}^{T} p(x_t \mid x_1, \ldots, x_{t-1}), \\ &= \prod_{t=1}^{T} p(x_t \mid x_{<t}), \end{aligned} \tag{1.1}$$

with the conventions that $p(x_1 \mid) := p(x_1)$ is just the (marginal) probability of the first token and $x_{<t} := x_1, ..., x_{t-1}$, so $p(x_t \mid x_1, \ldots, x_{t-1}) = p(x_t \mid x_{<t})$. Equation (1.1) is exact — no assumption has been made yet — but it reduces the problem to something that is well posed. Build a model of $p(x_t \mid x_{<t})$, i.e., of **the next token given everything before it**. Every model in these notes, from the crudest to a full GPT, is a way of parameterizing the distribution of these conditional probabilities.

This is what *autoregressive* means: the model consumes a prefix and predicts a distribution over the next symbol, one position at a time. Training and sampling are both organized around this conditional:

- **Training**: given real sequences from a corpus, adjust the model so that it assigns high probability to the token that actually follows any given prefix.
- **Generation**: sample $\hat{x}_t \sim p(\cdot \mid x_{<t})$, append it to the prefix, and repeat.

1.2 Measuring fit: negative log-likelihood and perplexity

To compare models — and to train them with gradient descent — we need a scalar loss. The natural one falls directly out of maximum likelihood. According to the chain rule of Eq. (1.1), the **average negative log-likelihood per token** of a given sequence $x_1, \ldots, x_T$ is

$$\mathcal{L} = -\frac{1}{T} \log p(x_1, \ldots, x_T) = -\frac{1}{T} \sum_{t=1}^{T} \log p(x_t \mid x_{<t}). \tag{1.2}$$

It is convenient to name the loss incurred at a single position t,

$$\ell_t := -\log p(x_t \mid x_{<t}), \tag{1.3}$$

so that $\mathcal{L} = \frac{1}{T} \sum_{t=1}^{T} \ell_t$ is simply its average over positions. This is exactly the **cross-entropy loss** between the model's predicted distribution and the **one-hot distribution** placing all mass on the observed token x_t, averaged over positions. worth spelling out. More specifically, for two distributions q and p over the same vocabulary $\{1, \ldots, V\}$, the **cross-entropy** of p relative to q is

$$H(q, p) = -\sum_{v=1}^{V} q(v) \log p(v). \tag{1.4}$$

At position t the model supplies one of the two distributions, namely its prediction for the next token given the prefix,

$$p_t(v) := p(x_t = v \mid x_{<t}), \qquad v = 1, \ldots, V, \tag{1.5}$$

and the corpus supplies the other. That is, the corpus tells us that the token that actually occurred at time t is x_t. Viewed as a distribution over the vocabulary — the empirical distribution of that single observation — this is the **one-hot** distribution (Kronecker delta)

$$q_t(v) = \delta_{v, x_t}. \tag{1.6}$$

Substituting Eqs. (1.5) and (1.6) into Eq. (1.4), the sum over the vocabulary collapses, because every term with $v \neq x_t$ is multiplied by zero:

$$\begin{aligned} H(q_t, p_t) &= -\sum_{v=1}^{V} \delta_{v, x_t} \log p_t(v) \\ &= -\log p_t(x_t) \\ &= -\log p(x_t \mid x_{<t}). \end{aligned} \tag{1.7}$$

Averaging Eq. (1.7) over the T positions of the sequence reproduces Eq. (1.2) term by term,

$$\frac{1}{T} \sum_{t=1}^{T} H(q_t, p_t) = -\frac{1}{T} \sum_{t=1}^{T} \log p(x_t \mid x_{<t}) = \mathcal{L}, \tag{1.8}$$

which is the claim: *average negative log-likelihood* and *average cross-entropy against one-hot targets* are two names for the same number.

Note that the one-hot vector q_t never has to be built in practice. The sum in Eq. (1.7) has a single surviving term, so all an implementation needs is the integer index x_t, with which to read off one entry of the predicted distribution.

The identity also pins down where the loss is minimized. Writing Eq. (1.4) in terms of the Shannon entropy $H(q) = -\sum_v q(v) \log q(v)$ and the Kullback–Leibler divergence $D_{\mathrm{KL}}(q \,\|\, p) = \sum_v q(v) \log \frac{q(v)}{p(v)}$ gives the standard decomposition

$$H(q, p) = H(q) + D_{\mathrm{KL}}(q \,\|\, p). \tag{1.9}$$

For a one-hot q_t the entropy term vanishes, $H(q_t) = 0$, so the loss is a pure average divergence between observations and model,

$$\mathcal{L} = \frac{1}{T} \sum_{t=1}^{T} D_{\mathrm{KL}}(q_t \,\|\, p_t) \;\geq\; 0, \tag{1.10}$$

where the inequality is Gibbs' inequality, with equality if and only if $p_t = q_t$. Lower is therefore better, and $\mathcal{L} = 0$ only if the model produces a perfect prediction, i.e. if it assigns probability 1 to every observed token. Minimizing $\mathcal{L}$ thus means pushing each predicted distribution as close as possible — in the KL sense — to the observation at that position. Since no model can place all its mass on the observed token at every position of a real corpus, the attainable minimum is strictly positive, and the useful question is always *how much* lower one model gets than another.

$\mathcal{L}$ is in units of *nats per token* (natural log) and is not very interpretable by itself. Its exponential,

$$\mathrm{PPL} = e^{\mathcal{L}}, \tag{1.11}$$

is called **perplexity** and has a more intuitive reading. **It is the effective number of choices the model is "confused between" at each step.** A model that always assigns uniform probability over the V tokens has $\mathcal{L} = \log V$ and $\mathrm{PPL} = V$, which is the maximum possible. Any useful model should do better than that. For the character-level vocabulary used in our experiments ($V = 65$), $\log V \approx 4.17$ nats, which is the loss value you should expect from an untrained model at initialization.

Remark (tokenization)

Real GPT models do not operate on characters; they operate on sub-word units produced by an algorithm such as byte-pair encoding (BPE), so that frequent words or word-fragments become a single token (GPT-2's tokenizer has a vocabulary of about 50,000 such tokens). This shortens sequences substantially (a given sentence is 3–4× shorter in BPE tokens than in characters) and lets the model spend its capacity on higher-level structure rather than on re-deriving spelling. We use a character-level tokenizer throughout these notes because it makes the vocabulary — and hence every table and matrix indexed by it — small enough to reason about by hand; nothing else in the architecture changes when the tokenizer changes.

Implementation note

`gpt_intro/data.py:CharTokenizer` is the character-level tokenizer used throughout: it maps each distinct character in the corpus to an integer id and back.

The next chapter turns to the question of how a model can represent the context $x_{<t}$ on which it conditions.

Chapter 2

Embedding the context

Chapter 1 reduced language modeling to a single task, building a model of the conditional distribution $p(x_t \mid x_{<t})$, and fixed the loss that measures how well a model does it, Eq. (1.2). Before committing to any particular architecture, this chapter describes the form that *every* model in these notes takes. The idea is simple, but everything that follows rests on it: a model is a machine that turns the context $x_{<t}$ into a vector, and the prediction for x_t is read off from that vector.

2.1 From contexts to vectors

A model of $p(x_t \mid x_{<t})$ must produce, for every possible context $x_{<t}$, a probability distribution over the V tokens of the vocabulary. Contexts are awkward objects to work with. They have variable length, they are discrete, and there are V^{t-1} of them at position t. Moreover, nothing in the raw sequence of token ids says which contexts are similar to each other.

When the set of contexts is small enough to be enumerated, a model can simply be a table, with one probability distribution stored per context. This is the case of the n-gram models of Chapter 3, where the context is truncated to its last $n-1$ tokens and there are V^{n-1} distinct contexts, a manageable number only for small n. In general, however, a table is out of the question: the number of contexts is far too large, and most of them never occur in any corpus (Section 3.3). Instead, we let the model summarize the context in a fixed-size vector of real numbers, the **context vector**

$$c_{t-1} \in \mathbb{R}^d, \tag{2.1}$$

computed from $x_1, \ldots, x_{t-1}$ by a learnable function. The context vector is an **embedding** of the context: a representation, in a vector space, of the information available before x_t. Since nothing else reaches the prediction, everything the model knows about the past that is relevant to predicting x_t must be encoded in c_{t-1}. Representing contexts as vectors has a further advantage over a table. Contexts that should lead to similar predictions can be mapped to nearby vectors, so that what the model learns from one context carries over to the others, including contexts it has never seen during training.

The indices in Eq. (2.1) follow a convention that we keep for the rest of these notes. A representation is indexed by the *last token it has read*: c_{t-1} summarizes $x_1, \ldots, x_{t-1}$. A logit vector, introduced in the next section, is instead indexed by the *token it predicts*.

All the architectures in these notes have this form, and they differ *only* in how c_{t-1} is computed:

- in the **bigram** model (Chapter 3), c_{t-1} depends only on x_{t-1}, and is read off a lookup table;
- in a **bag-of-words** model (Section 3.6), c_{t-1} is the uniform average of the embeddings of all the tokens of the context;

- with **self-attention** (Chapter 4), the average becomes learned and content-dependent;
- in **GPT** (Chapter 6), the context vector is the output of many layers of attention and feed-forward networks applied to the whole prefix.

Everything else — how the context vector is turned into a distribution, the loss, the training loop and the sampling procedure — is shared by all of them, and is the subject of the rest of this chapter.

Note that c_{t-1} may depend on the tokens before position t only, never on x_t or on any later token. Otherwise the prediction of x_t would be trivial during training and impossible during generation, when the future has not been produced yet. For the bigram model this holds by construction. For attention, it has to be enforced explicitly, with a causal mask (Section 4.3).

2.2 Logits and softmax

The context vector lives in $\mathbb{R}^d$, while the prediction must be a probability distribution over the V tokens. We could let the model output a vector of probabilities directly. However, its entries would have to be non-negative and sum to one. These are two constraints that are difficult to enforce during gradient descent. Therefore, we let a learned linear map turn the context vector into an unconstrained vector of **logits**,

$$z_t = W c_{t-1} \in \mathbb{R}^V, \qquad W \in \mathbb{R}^{V \times d}, \tag{2.2}$$

which scores every candidate for x_t. Following the convention of Section 2.1, z_t is indexed by the token it predicts, hence the shift by one in Eq. (2.2). Two different objects, playing two different roles, are kept separate: the context vector *summarizes* the prefix in $\mathbb{R}^d$, and the logit vector *scores* every candidate next token in $\mathbb{R}^V$.

A vector of logits $z \in \mathbb{R}^V$ is turned into a probability distribution with the **softmax** function $\sigma : \mathbb{R}^V \to \Delta^{V-1}$,

$$\sigma_b(z) := \frac{e^{z_b}}{\sum_{b'} e^{z_{b'}}}, \qquad b = 1, \dots, V, \tag{2.3}$$

whose codomain is the **probability simplex** $\Delta^{V-1} := \{\pi \in \mathbb{R}^V : \pi_b \geq 0,\ \sum_b \pi_b = 1\}$, the set of probability distributions over the V tokens. Softmax is non-negative, sums to one by construction, is differentiable and is shift-invariant ($\sigma(z) = \sigma(z + \kappa \mathbf{1})$ for any $\kappa \in \mathbb{R}$, where $\mathbf{1} = (1, 1, \dots, 1)^T \in \mathbb{R}^V$). Hence, any real-valued vector of logits maps to a valid probability distribution, without the need of introducing a normalization constraint. The model's prediction for the token at position t is

$$p_\theta(x_t = b \mid x_{<t}) = \sigma_b(z_t) = \frac{e^{(z_t)_b}}{\sum_{b'} e^{(z_t)_{b'}}}, \tag{2.4}$$

where θ denotes the collection of all the learnable parameters of the model, those that compute c_{t-1} together with W. Because of shift invariance, the logits are determined by the predicted distribution only up to an additive constant.

2.3 The loss in terms of logits

Substituting Eq. (2.4) into the per-position loss of Eq. (1.3) gives

$$\ell_t(\theta) = -\log \sigma_{x_t}(z_t) = -(z_t)_{x_t} + \log \sum_b e^{(z_t)_b}. \tag{2.5}$$

This is the **cross-entropy** of Eq. (1.4) between the one-hot distribution on the observed token x_t and the predicted distribution $\sigma(z_t)$. Note that Eq. (2.5) depends on the model only through

z_t, and makes no reference to how z_t was computed. This is the form of the loss we use for every architecture in these notes. Its gradient with respect to the logits is particularly simple,

$$\frac{\partial \ell_t}{\partial (z_t)_b} = \sigma_b(z_t) - \delta_{b,x_t}. \tag{2.6}$$

That is, gradient descent raises the logit of the observed token and lowers every other logit in proportion to the probability the model currently assigns to it. The gradient vanishes only when the model assigns probability one to the observed token. The total loss $\mathcal{L}(\theta)$ is the average of $\ell_t(\theta)$ over all positions, Eq. (1.2), and it is minimized over θ by gradient descent, with the gradients with respect to θ computed by automatic differentiation (Chapter 7).

Implementation note

`F.cross_entropy` computes the loss $\mathcal{L}$, i.e. the average of Eq. (2.5) over all positions in a batch. It takes raw logits (not probabilities) and, since the one-hot targets never have to be built (Eq. (1.7)), target *indices* rather than target distributions. It applies Eqs. (2.3) and (2.5) internally, in a numerically stable way (subtracting $\max_b (z_t)_b$ before exponentiating, which Eq. (2.5) shows does not change the result).

2.4 Token embeddings

The input of the model is a sequence of integer token ids, and the first step of every architecture in these notes is to map each id to a learnable vector. This is done with an **embedding table** $E \in \mathbb{R}^{V \times d}$. We write $e_a \in \mathbb{R}^d$ for row a of E, the **token embedding** of token a, so that the token x_s at position s is mapped to the vector e_{x_s}. Looking up token a returns row a. There is no matrix multiplication involved in the lookup itself (it is indexing), but the table is still an ordinary learnable parameter tensor, differentiated through by autograd exactly like the weight matrix of a linear layer. Equivalently, $e_a = E^\top u_a$, where $u_a \in \mathbb{R}^V$ is the one-hot vector of token a, so a lookup is a linear layer applied to a one-hot input, implemented efficiently. The embedding dimension d is a design choice, independent of V.

With token embeddings at hand, every model in these notes has the same three-stage structure,

$$x_1, \ldots, x_{t-1} \;\longmapsto\; e_{x_1}, \ldots, e_{x_{t-1}} \;\longmapsto\; c_{t-1} \in \mathbb{R}^d \;\longmapsto\; z_t = W c_{t-1} \in \mathbb{R}^V. \tag{2.7}$$

The first stage embeds each token separately. The second combines the token embeddings into the context vector c_{t-1}. The third projects c_{t-1} onto the space of logits, Eq. (2.2). The first and the last stage are the same for all the architectures, and the chapters that follow are almost entirely about the middle one.

We therefore use the word "embedding" for two different objects: the embedding e_a of a single token, and the embedding c_{t-1} of a whole context. In the bigram model, the simplest case of Eq. (2.7), they coincide: the context vector is just the token embedding of x_{t-1}, with $d = V$, used directly as logits (Section 3.5).

Implementation note

`nn.Embedding(V, d)` is the embedding table E: its `weight` is a $V \times d$ parameter tensor, and calling it on a tensor of token ids returns the corresponding rows. The projection W of Eq. (2.2) is an `nn.Linear(d, V)`.

Chapter 3

n-gram and bigram models

This chapter studies the simplest models of language: those that keep only the last few tokens of the context. For a fixed number of tokens kept, the contexts can be enumerated, and the model can be a plain table of conditional probabilities, fitted exactly by counting. We state the counting solution for a general context length, and then examine why it fails as the context grows: the table becomes exponentially large and mostly empty, and what is learned about one context says nothing about any other. These two failures are what the context vectors of Chapter 2, and every architecture after this chapter, are designed to overcome.

We then specialize to the shortest non-trivial context, a single token, which gives the bigram model. We fit the bigram model in two ways, first by counting and then by gradient descent, and check that both reach the same loss. Counting gives the maximum-likelihood solution in closed form only because every context owns an independent row of the table. Once parameters are shared across contexts, as in all the models from Chapter 4 on, there is no closed form, while gradient descent carries over unchanged. Fitting the bigram model by gradient descent is therefore what makes the transition to attention a change of architecture rather than a change of training procedure.

3.1 Truncating the context: n-gram models

Equation (1.1) conditions on the *entire* history $x_{<t}$, which is exactly the part that is hard, because the number of distinct histories grows exponentially with t. The n-**gram** approximation keeps only the last $n-1$ tokens of the history,

$$p(x_t \mid x_1, \ldots, x_{t-1}) \;\approx\; p(x_t \mid x_{t-n+1}, \ldots, x_{t-1}), \tag{3.1}$$

so that the context has a fixed length $n-1$, and n is the length of the window formed by the context and the predicted token. This is the assumption that the sequence of tokens is a Markov chain of order $n-1$. The cases $n = 1$, 2 and 3 are called the **unigram**, **bigram** and **trigram** models. The unigram model ignores the context altogether and predicts every token with the same distribution, the frequency of each token in the corpus.

For a fixed n, an n-gram model is a table. Write $a_{<n} := (a_1, \ldots, a_{n-1})$ for a context, i.e. for a sequence of $n-1$ tokens of the vocabulary, and a_n for the token that follows it. The model is specified by the conditional probabilities

$$P_{a_{<n}a_n} := p(x_t = a_n \mid x_{t-n+1} = a_1, \ldots, x_{t-1} = a_{n-1}), \tag{3.2}$$

which do not depend on t. For each of the V^{n-1} possible contexts $a_{<n}$, the row of entries $P_{a_{<n}a_n}$ for $a_n \in \{1, \ldots, V\}$ is a probability distribution over the next token, so the table has $V^{n-1}(V-1)$

free parameters. Every context has its own row, chosen independently of the rows of all the other contexts.

The first $n-1$ tokens of a sequence do not have a full context. They can be handled with lower-order models, or by padding the sequence with a special start token. To keep the notation simple, we just leave them out and evaluate the model on positions $t = n, \dots, T$ only.

3.2 Fitting an n-gram model by counting

Given a training corpus $x_1, \dots, x_T$, let

$$N_{a_{<n}a_n} = \sum_{t=n}^{T} \prod_{k=1}^{n} \delta_{x_{t-n+k},\, a_k} \tag{3.3}$$

be the number of times the sequence of n tokens $a_1, \dots, a_n$ occurs in the corpus, where

$$\delta_{x,y} = \begin{cases} 1, & x = y, \\ 0, & x \neq y \end{cases}$$

is Kronecker's delta. Summing over the last index gives the number of times the context $a_{<n}$ occurs followed by some token,

$$N_{a_{<n}} := \sum_{a_n=1}^{V} N_{a_{<n}a_n}, \qquad \sum_{a_{<n}} N_{a_{<n}} = T - n + 1. \tag{3.4}$$

The **maximum-likelihood estimate** of an n-gram model is

$$\hat{P}_{a_{<n}a_n} = \frac{N_{a_{<n}a_n}}{N_{a_{<n}}} = \frac{N_{a_{<n}a_n}}{\sum_{a'_n} N_{a_{<n}a'_n}}, \tag{3.5}$$

that is, the observed frequency of a_n after $a_{<n}$. We derive Eq. (3.5) for the bigram case in Section 3.4. The derivation carries over to any n unchanged, because it only uses that every context has its own row of the table, independent of all the others. Two difficulties remain open in Eq. (3.5). For a context that never occurs in the corpus, $N_{a_{<n}} = 0$ and the estimate of Eq. (3.5) is undefined. For a context that occurs, every token never observed after it gets probability exactly zero, so a single such occurrence in new text makes the loss infinite. The standard remedy is additive (Laplace) **smoothing**, which adds a constant $\nu > 0$ to every count before normalizing,

$$\hat{P}_{a_{<n}a_n} = \frac{N_{a_{<n}a_n} + \nu}{N_{a_{<n}} + V\nu}. \tag{3.6}$$

The $V\nu$ in the denominator, one ν for each of the V entries of the row, keeps every row normalized. An unseen context now gets the uniform distribution, and an unseen continuation of a seen context gets a small positive probability.

3.3 Limitations of counting

Increasing n makes the approximation of Eq. (3.1) better: more of the history is used, and in the limit $n \to T$ nothing is truncated. In practice, however, counting runs into the following problems long before that limit.

Exponential size. The table has V^{n-1} rows, a number that grows exponentially with n. For our character-level vocabulary, $V = 65$, this is 65 rows for $n = 2$, about 2.7×10^5 for $n = 4$ and about 1.2×10^9 for $n = 6$. Storage alone is not the real obstacle, since at most $T - n + 1$ entries of N are nonzero, and a sparse data structure stores only those. The obstacle is what those counts can tell us.

Sparsity. The corpus contains at most $T - n + 1$ windows, while the number of possible contexts grows exponentially with n. For large n almost every possible context is never observed, and most of the observed ones are observed only once or a few times. On Tiny Shakespeare, for instance, we fit the counts on the first 90% of the corpus, the *training split*, and keep the last 10%, the *validation split*, to measure how well the model predicts text it has not seen (Section 7.1). Already for $n = 3$ only a third of the possible contexts occur in the training split, and for $n = 5$ fewer than 0.3% do. An estimate built from one or two occurrences is extremely noisy, and an unobserved context gets no estimate at all.

Overfitting. Because the rows are fitted independently, a model with large n can reproduce the training corpus very closely. On Tiny Shakespeare the unsmoothed training loss falls steadily with n, from 3.31 nats for the unigram to 0.76 for $n = 7$. The validation loss of the smoothed model, on the contrary, reaches its minimum, about 1.95 nats, at $n = 4$, and grows after that, up to about 3.0 nats for $n = 7$. This is a bias–variance trade-off. A small n has a large bias, the error of the truncation in Eq. (3.1), but low variance, since every row is estimated from many occurrences. A large n has a small bias, but every row is estimated from a handful of occurrences, so the estimates fit the accidents of the training corpus instead of the language. Laplace smoothing does not resolve the trade-off: with $\nu = 1$ and $V = 65$, a context seen only a few times gets a row dominated by the 65 pseudo-counts, which is nearly uniform.

No generalization across contexts. The deepest limitation is structural. In a table, every context is an atom: nothing in the row of one context depends on what was observed after any other context. At the word level, having seen "the cat sat on the" many times says nothing about what follows "the dog sat on the", although the two contexts differ by one word and play the same role. At the character level, what is learned after "king" says nothing about what follows "King". Neural language models were introduced precisely to address this (Bengio et al., 2003): represent each token by a learned vector and compute the prediction from these vectors, so that similar contexts are mapped to similar predictions.

Fixed horizon. Whatever n is, nothing that happened more than $n - 1$ tokens ago can influence the prediction. Note that the GPT of Chapter 6 also has a fixed horizon, its context length $L_{\max}$, so formally it is a Markov chain of order $L_{\max}$ too. For the demo configuration, $L_{\max} = 128$ and a table would need $65^{128} \approx 10^{232}$ rows, while the GPT has about 8×10^5 parameters (Section 6.4). What separates the two models is not how much of the context they see, but how the dependence on the context is parameterized: an independent row per context in one case, a single function shared by all the contexts in the other.

3.4 The case $n = 2$: bigrams

The bigram model, $n = 2$, truncates the context to a single token,

$$p(x_t \mid x_1, \ldots, x_{t-1}) \;\approx\; p(x_t \mid x_{t-1}). \tag{3.7}$$

Writing $a = a_1$ for the previous token and $b = a_2$ for the next one, the table of Eq. (3.2) becomes a $V \times V$ matrix P, with entries $P_{ab} = p(x_t = b \mid x_{t-1} = a)$. Row a of P is the distribution of the token that follows a. The counts of Eq. (3.3) become

$$N_{ab} = \sum_{t=2}^{T} \delta_{x_{t-1},a}\, \delta_{x_t,b}, \tag{3.8}$$

the number of times the pair $(x_{t-1}, x_t) = (a, b)$ occurs in the corpus. Let $N_a := \sum_b N_{ab}$ be the number of times token a occurs followed by some token.

The maximum-likelihood estimate follows from grouping the positions of the corpus by the pair of tokens that occurs at them. Each pair (a, b) contributes the same term $\log P_{ab}$ every time it occurs, so the log-likelihood of the corpus under the model is

$$\sum_{t=2}^{T} \log P_{x_{t-1}x_t} = \sum_a \sum_b N_{ab} \log P_{ab} = \sum_a N_a \sum_b \hat{q}_a(b) \log P_{ab}, \tag{3.9}$$

where $\hat{q}_a(b) := N_{ab}/N_a$ is the empirical distribution of the tokens that follow a in the corpus. The inner sum is minus the cross-entropy $H(\hat{q}_a, P_a)$, Eq. (1.4), between $\hat{q}_a$ and row a of the table, P_a. The log-likelihood therefore splits into independent terms, one per row, and each row can be optimized separately. By the decomposition $H(\hat{q}_a, P_a) = H(\hat{q}_a) + D_{\mathrm{KL}}(\hat{q}_a \,\|\, P_a)$ of Eq. (1.9) and Gibbs' inequality, the term of row a is maximized when $P_a = \hat{q}_a$. The maximum-likelihood estimate is thus

$$\hat{P}_{ab} = \hat{q}_a(b) = \frac{N_{ab}}{\sum_{b'} N_{ab'}}, \tag{3.10}$$

the case $n = 2$ of Eq. (3.5). At this estimate the KL terms vanish, and the average loss of Eq. (1.2) on the training corpus is

$$\mathcal{L} = \sum_a \frac{N_a}{T-1} H(\hat{q}_a), \tag{3.11}$$

the entropy of the next token averaged over the previous token, i.e. the conditional entropy of x_t given x_{t-1}, measured on the corpus. It is the uncertainty about the next token that remains once the previous one is known, and no bigram model can do better on the training corpus. For a general n, the same steps give the conditional entropy of x_t given its $n-1$ predecessors.

With the smoothing of Eq. (3.6),

$$\hat{P}_{ab} = \frac{N_{ab} + \nu}{\sum_{b'} N_{ab'} + V\nu}.$$

The code uses $\nu = 1$.

Implementation note

`gpt_intro/bigram.py:CountingBigram` builds exactly this table: `counts[a, b]` accumulates co-occurrences, and `.probs` normalizes each row. Generalizing it to an n-gram model, indexing the counts by tuples of $n-1$ token ids, is a short exercise, and reproduces the numbers quoted in Section 3.3.

For $n = 2$ the limitations of Section 3.3 are at their mildest. With $V = 65$ there are only 65 contexts, all of them occur in the corpus, and each one occurs on average about 1.5×10^4 times, so every row is estimated reliably. The training and validation losses, 2.45 and 2.48 nats, are nearly equal: the bigram model hardly overfits. Its problem is the opposite one, the bias of Eq. (3.7). It is

a first-order Markov chain over characters, and it produces text that is locally plausible but globally incoherent. It cannot even spell: after "th" its prediction depends only on the "h", exactly as after "sh" or "wh". Nothing that happened more than one token ago can influence the next prediction.

Remark

The counting bigram reaches a training loss around 2.45 nats (perplexity ≈ 11.6) on Tiny Shakespeare, i.e., it is on average as uncertain as choosing between about 12 equally likely characters – much better than guessing uniformly among all 65, but still far from a good model, since it can never use more than one character of context. This loss and perplexity are obtained by running `code/scripts/train_bigram.py`.

We study the bigram model in detail anyway, because it already contains, in miniature, every ingredient we will need: a context vector, a loss that measures how well the model fits the data, and a way to sample from it. Chapters 4 onward are entirely about *removing* the truncation in Eq. (3.7) while avoiding the limitations of Section 3.3, and keeping the rest of the machinery.

3.5 The same model, trained by gradient descent

Counting, Eq. (3.10), is the exact maximum-likelihood solution for a bigram model, so there is no accuracy to be gained by training it with gradient descent instead. We do it anyway, for the reasons given at the beginning of this chapter.

In the language of Chapter 2, the bigram approximation says that the context vector depends only on the last token read. The simplest choice is to take the context vector to be the token embedding of that token, $c_{t-1} = e_{x_{t-1}}$, with $d = V$, and $W = I$ in Eq. (2.2), so that the token embedding is used directly as logits. To emphasize this role, we call the embedding table $Z \in \mathbb{R}^{V \times V}$ instead of E, with rows $Z_a \in \mathbb{R}^V$ and entries $Z_{ab} := (Z_a)_b$. Then

$$z_t = Z_{x_{t-1}}, \tag{3.12}$$

that is, the logits for x_t are row x_{t-1} of Z, and by Eq. (2.4) the model's estimate of the bigram table is

$$p_Z(x_t = b \mid x_{t-1} = a) = \sigma_b(Z_a) = \frac{e^{Z_{ab}}}{\sum_{b'} e^{Z_{ab'}}}. \tag{3.13}$$

In other words, row Z_a is an unconstrained re-parameterization of row a of the matrix P of Section 3.4.

Because there are only V possible contexts, we can tabulate the loss in advance. For every pair of tokens $a, b \in \{1, \dots, V\}$, the loss the model would incur by predicting token b right after context a is

$$\ell_{ab}(Z) := -\log \sigma_b(Z_a) = -Z_{ab} + \log \sum_{b'} e^{Z_{ab'}}. \tag{3.14}$$

This is a fixed $V \times V$ array of numbers, computable from the model's parameters Z_{ab} alone, with no reference yet to any actual corpus. At position t of a specific sequence, the loss is Eq. (3.14) evaluated at the pair that actually occurs there,

$$\ell_t(Z) = \ell_{x_{t-1}, x_t}(Z) = -\log p_Z(x_t \mid x_{t-1}), \tag{3.15}$$

which is Eq. (2.5) with $z_t = Z_{x_{t-1}}$, and the ℓ_t of Eq. (1.3) with the context $x_{<t}$ truncated to x_{t-1}. The table ℓ_{ab} exists only because the contexts can be enumerated, and it is useful only while they are few. Equation (2.5), instead, carries over unchanged to every model.

As a function of the table P itself (subject to each row summing to one), the loss of Eq. (1.2) is minimized *exactly* by the counting solution Eq. (3.10) in the limit of no smoothing, as shown in Section 3.4. Since the softmax maps $\mathbb{R}^V$ onto every distribution with strictly positive entries, minimizing over Z can approach the same minimum. A correctly trained neural bigram model should therefore reach the same loss as the counting model, which is why the counting model is a useful sanity check.

Note that the context vector generalizes the embedding $e_{x_{t-1}}$ of the previous token, not the logit vector z_t. The two can be identified in the bigram model only because there the embedding is used as logits directly.

The neural parameterization by itself does not remove any of the limitations of Section 3.3. A neural n-gram model built in the same way, with one row of logits per context of $n-1$ tokens, would be a $V^{n-1} \times V$ lookup table, with the same sparsity and the same lack of generalization as the table of counts. What helps is to stop treating the context as a single atom to be looked up, and to build its representation from the embeddings of its individual tokens, which are shared by all the contexts in which the tokens appear. This is the subject of the next section.

Implementation note

`gpt_intro/bigram.py:NeuralBigramLM` is a single `nn.Embedding(vocab_size, vocab_size)`, whose weight is the matrix Z above (autograd differentiates through it exactly like the weight of an `nn.Linear`): `forward` looks up the row for each input token and returns it as the logits for the next token, and `F.cross_entropy` is applied against the targets. There is no hidden layer and no nonlinearity — this model has exactly as much representational capacity as the counting table of Section 3.4, which is why `scripts/train_bigram.py` trains both and expects them to reach the same loss.

3.6 The real limitation, and what a fix would look like

However it is trained, a bigram model can only ever look at x_{t-1}: two prefixes that end in the same character but differ arbitrarily before that are treated identically. Increasing n is the answer of counting, and Section 3.3 showed where it leads. What we actually want is a context vector c_{t-1} that summarizes the *whole* prefix $x_1, \ldots, x_{t-1}$, built from the d-dimensional embeddings of its tokens, and that Eq. (2.2) then turns into logits for x_t. Since the embeddings are shared, two contexts that contain the same tokens share parameters, even if neither of them was ever observed in full.

The crudest way to use the whole prefix is to average the embeddings of all the tokens read so far, $c_{t-1} = \frac{1}{t-1} \sum_{s \le t-1} e_{x_s}$ ("bag of words" up to position $t-1$). Through Eq. (2.2), and since W is linear,

$$z_t = \frac{1}{t-1} \sum_{s \le t-1} W e_{x_s}, \tag{3.16}$$

i.e. the logits are the average of the logit vectors We_{x_s} that each previous token would produce on its own, as if it were the only context in a bigram model. This already uses the whole history, with only $2Vd$ parameters however long the history is, but every past token contributes with the *same* fixed weight $\frac{1}{t-1}$ regardless of content and of position — the word "not" three tokens back should plausibly matter more to the next prediction than a comma five tokens back, and a uniform average cannot express that. What is missing is a way to make the averaging weights *learned* and *content-dependent*: weights that the model computes on the fly from the actual embeddings at hand,

instead of weights fixed in advance. That is precisely what self-attention is, and it is the subject of the next chapter.

Chapter 4

Self-attention

4.1 Queries, keys and values

Section 3.6 left us wanting a context vector c_t, summarizing $x_1, \dots, x_t$, built from a *learned, content-dependent* weighted average of the embeddings at positions $s \le t$. Self-attention builds exactly this, using three learned linear projections of the embedding $e_{x_s} \in \mathbb{R}^d$ of every token:

$$q_s = W_Q^\top e_{x_s} \in \mathbb{R}^{d_k} \qquad \text{(query: "what am I looking for?")} \tag{4.1}$$

$$k_s = W_K^\top e_{x_s} \in \mathbb{R}^{d_k} \qquad \text{(key: "what do I contain?")} \tag{4.2}$$

$$v_s = W_V^\top e_{x_s} \in \mathbb{R}^{d_k} \qquad \text{(value: "what do I offer, if attended to?")} \tag{4.3}$$

with $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ learned matrices (no bias, following GPT-2). Every position produces all three vectors, using the *same* three matrices — this is what makes attention a single mechanism applied uniformly across a variable-length sequence, rather than one set of parameters per position.

The query at position t is compared against the keys at every position $s \le t$ via a dot product $q_t \cdot k_s$: a large dot product means "key s matches what query t is looking for", and should translate into a large weight on value v_s in the output at position t. Turning these dot products into normalized, non-negative weights is exactly what softmax does (Eq. (2.3)), so the context vector we wanted is

$$c_t = \sum_{s \le t} \alpha_{ts}\, v_s, \qquad \alpha_{ts} = \sigma_{s \le t}\!\left(\frac{q_t \cdot k_s}{\sqrt{d_k}}\right). \tag{4.4}$$

Unlike the uniform average of Section 3.6, the weights α_{ts} now depend on the actual content of e_{x_t} and e_{x_s} through q_t and k_s, and are learned end to end along with every other parameter in the model.

This already closes the chain from tokens to logits: feeding the context vector of Eq. (4.4) to the linear map of Eq. (2.2),

$$x_1, \dots, x_t \;\longrightarrow\; e_{x_s} \;\longrightarrow\; q_s, k_s, v_s \;\longrightarrow\; c_t \;\longrightarrow\; z_{t+1} = W c_t, \qquad W \in \mathbb{R}^{V \times d_k}, \tag{4.5}$$

is a complete language model: a single attention layer between the embeddings and the logits of Eq. (2.7), with z_{t+1} scoring the candidates for the next token x_{t+1}.

4.2 Why the $\sqrt{d_k}$

If the components of q_t and k_s were independent random variables with mean 0 and variance 1, the dot product $q_t \cdot k_s = \sum_{i=1}^{d_k} q_{ti} k_{si}$ would have mean 0 and *variance* d_k (variance of a sum of

d_k independent terms of unit variance). For anything but small d_k, the raw dot products would therefore have a standard deviation of order $\sqrt{d_k}$, pushing softmax into a saturated regime where one logit dominates and the rest receive gradients that vanish. Dividing by $\sqrt{d_k}$ rescales the dot products back to unit variance, keeping softmax in a well-conditioned regime regardless of d_k; this is exactly the scaling in Eq. (4.4), and the reason the mechanism is called *scaled* dot-product attention (Vaswani et al., 2017).

4.3 Matrix form and the causal mask

Stack the embeddings $e_{x_1}, \dots, e_{x_T}$ of a length-T sequence as the rows of a matrix $H \in \mathbb{R}^{T \times d}$, giving $Q = HW_Q$, $K = HW_K$, $V = HW_V$, each in $\mathbb{R}^{T \times d_k}$. Then Eq. (4.4) for every position at once is

$$C := \begin{bmatrix} c_1^\top \\ \vdots \\ c_T^\top \end{bmatrix} = \mathcal{A}(Q, K, V) = \sigma\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V \in \mathbb{R}^{T \times d_k}, \tag{4.6}$$

where softmax is applied row-wise, so that row t of the right-hand side is $\sum_{s \le t} \alpha_{ts} v_s^\top = c_t^\top$. Equation (4.6) therefore introduces no new representation: it computes all T context vectors of Eq. (4.4) at once, one per row of C, just as H, Q, K and V hold one position per row. (Vectors such as q_s are columns, which is why stacking them as rows produces the transposes, and why $Q = HW_Q$ is consistent with $q_s = W_Q^\top e_{x_s}$.)

The map

$$\mathcal{A} : \mathbb{R}^{T \times d_k} \times \mathbb{R}^{T \times d_k} \times \mathbb{R}^{T \times d_k} \to \mathbb{R}^{T \times d_k}, \qquad (Q, K, V) \mapsto \mathcal{A}(Q, K, V), \tag{4.7}$$

defined by Eq. (4.6) is the **attention function**, written Attention(Q, K, V) in the literature. It has no parameters of its own: the learned parameters W_Q, W_K, W_V enter only through its arguments. It is also the only operation of the transformer that mixes positions: row t of $\mathcal{A}(Q, K, V)$ depends on all the rows $s \le t$ of K and V.

The matrix $M \in \{0, -\infty\}^{T \times T}$ is the **causal mask**: $M_{ts} = 0$ if $s \le t$ and $M_{ts} = -\infty$ if $s > t$. Adding $-\infty$ before softmax sends the corresponding weight to exactly 0 (Figure 4.1), which is what enforces the autoregressive property from Chapter 1: position t must never have access to positions after it, neither during training (where the whole sequence is available at once, see Section 7.1) nor during generation (where later positions do not exist yet).

Implementation note

`gpt_intro/attention.py:CausalSelfAttentionHead` implements Eq. (4.6) directly: three `nn.Linear` layers (without bias) for W_Q, W_K, W_V; `q @ k.transpose(-2,-1)` for $QK^\top$; a lower-triangular buffer `tril` (built once with `torch.tril`, not a trained parameter) to apply the mask via `masked_fill`; and `F.softmax` followed by `weights @ v`.

4.4 Computational cost

Computing $QK^\top$ is a $(T \times d_k) \times (d_k \times T)$ product, costing $O(T^2 d_k)$ time and, more importantly, producing a $T \times T$ matrix of scores that must be materialized (at least conceptually) before the row-wise softmax — $O(T^2)$ memory. This quadratic dependence on sequence length T is the central scalability issue of the transformer architecture: doubling the context length quadruples the

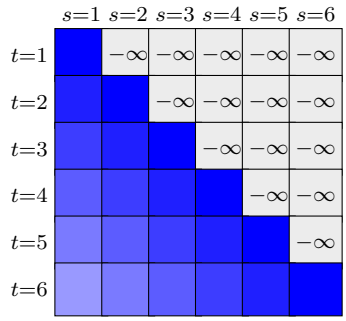


Figure 4.1: The causal mask before softmax, for $T = 6$. Row t, column s holds the score $q_t \cdot k_s/\sqrt{d_k}$ if $s \leq t$ (shaded by position, darker = more recent) and $-\infty$ if $s > t$ (grayed out): after row-wise softmax, every masked entry becomes an attention weight of exactly 0, so query t only ever mixes values from positions $1, \ldots, t$.

attention cost, whereas the rest of the model (embeddings, the feed-forward network in the next chapter) scales linearly in T. We will not address this further in these introductory notes — it is the motivation for techniques such as FlashAttention (fusing the softmax and matrix multiplications to avoid materializing the full $T \times T$ matrix) and sparse or linear attention variants, which become relevant once training moves to the cluster and to longer contexts.

Chapter 5

The transformer block

Chapter 4 ended with a complete model, Eq. (4.5), in which a single attention layer sits between the embeddings and the logits. This chapter inserts more processing in between. From now on, the vector representing position t is no longer just the embedding e_{x_t}, but a vector $h_t \in \mathbb{R}^d$ that is *updated in place* as it flows through the network, starting from $h_t = e_{x_t}$ (refined with positional information in Chapter 6). Following the convention of Section 2.1, h_t is indexed by the last token it has read: thanks to the causal mask, it depends only on $x_1, \dots, x_t$. The vectors $h_1, \dots, h_T$ are the rows of the matrix $H \in \mathbb{R}^{T \times d}$ fed to attention, which in Chapter 4 held the raw embeddings.

5.1 Multi-head attention

A single attention head (Section 4.1) computes *one* notion of relevance between positions, using one set of projections W_Q, W_K, W_V. There is no reason to expect that a single such notion suffices: one head might learn to attend to the immediately preceding token, another to the most recent noun, another to matching delimiters, and so on. **Multi-head attention** runs n_h attention heads in parallel, each with its own projections into a smaller space of dimension $d_k = d/n_h$, and combines their outputs. Head i applies the attention function $\mathcal{A}$ of Eq. (4.7) to its own queries, keys and values,

$$C^{(i)} = \mathcal{A}(HW_Q^{(i)}, HW_K^{(i)}, HW_V^{(i)}) \in \mathbb{R}^{T \times d_k}, \qquad W_Q^{(i)}, W_K^{(i)}, W_V^{(i)} \in \mathbb{R}^{d \times d_k}, \tag{5.1}$$

and **multi-head attention** (MHA) is the map $\mathcal{M} : \mathbb{R}^{T \times d} \to \mathbb{R}^{T \times d}$,

$$\mathcal{M}(H) = \left[C^{(1)} \,\|\, C^{(2)} \,\|\, \cdots \,\|\, C^{(n_h)} \right] W_O, \tag{5.2}$$

where $\|$ denotes concatenation along the feature dimension (giving back a $T \times n_h d_k = T \times d$ matrix) and $W_O \in \mathbb{R}^{d \times d}$ is one more learned linear projection that lets the heads' outputs mix before being passed on. Like $\mathcal{A}$, the map $\mathcal{M}$ mixes positions, and only causally: row t of $\mathcal{M}(H)$ depends on the rows $h_1, \dots, h_t$ of H only. Splitting the model dimension d across n_h heads of size $d_k = d/n_h$, rather than using n_h full-size heads, keeps the total number of parameters and the total compute of Eq. (5.2) comparable to a single head of size d — the point of using several heads is representational diversity, not more capacity per head.

Implementation note

`gpt_intro/attention.py` provides two implementations of Eq. (5.2) that compute the same function: `MultiHeadAttentionNaive` instantiates n_h separate `CausalSelfAttentionHead` modules and concatenates their outputs with a Python-level loop, matching Eq. (5.2) almost literally.

`MultiHeadAttentionFast` instead uses a single `nn.Linear(n_embd, 3*n_embd)` to compute Q, K, V for *all* heads at once and reshapes the result so that the head index becomes an extra batch dimension consumed by a single batched matrix multiplication — no Python loop over heads. This is the form used in nanoGPT and GPT-2 and is the one that matters for GPU throughput, which is why it is the one used inside `Block` below; the naive version exists so the two can be compared directly (they produce the same output shape and, with matching weights, the same values).

5.2 LayerNorm

Deep networks train more reliably when the scale of activations flowing through each layer stays controlled. **Layer normalization** standardizes each token's representation independently, across the feature dimension. Layer normalization is the map $\mathcal{N} : \mathbb{R}^d \to \mathbb{R}^d$,

$$\mathcal{N}(h) = \gamma \odot \frac{h - \mu(h)\,\mathbf{1}}{\sqrt{\mathrm{Var}(h) + \epsilon}} + \beta, \tag{5.3}$$

where $\mathbf{1} = (1, \ldots, 1)^\top \in \mathbb{R}^d$ and

$$\mu(h) = \frac{1}{d}\sum_{j=1}^{d} h_j, \qquad \mathrm{Var}(h) = \frac{1}{d}\sum_{j=1}^{d} \big(h_j - \mu(h)\big)^2 \tag{5.4}$$

are the mean and variance of the d components of h. Both are *scalars*, computed separately for every token, at every position and every element of the batch: $h - \mu(h)\mathbf{1}$ subtracts the same number from every component of h, and the division by $\sqrt{\mathrm{Var}(h) + \epsilon}$ divides every component by the same number. The constant ϵ is small, for numerical stability, and $\gamma, \beta \in \mathbb{R}^d$ are learned per-feature scale and shift parameters that let the network undo the normalization if that turns out to be useful. The symbol $\odot$ denotes the component-wise (Hadamard) product, $(u \odot v)_i = u_i v_i$, so in components

$$\big[\mathcal{N}(h)\big]_i = \gamma_i \frac{h_i - \mu(h)}{\sqrt{\mathrm{Var}(h) + \epsilon}} + \beta_i, \qquad i = 1, \ldots, d. \tag{5.5}$$

The statistics $\mu(h)$ and $\mathrm{Var}(h)$ are the same for every component i, while γ_i and β_i depend on i. Applied to a matrix $H \in \mathbb{R}^{T \times d}$, $\mathcal{N}(H)$ means $\mathcal{N}$ applied to each row separately, so $\mathcal{N}$ never mixes positions. This is why LayerNorm is a genuine building block on the same footing as a linear layer rather than something to derive: the normalization itself has no free parameters worth deriving, and γ, β are trained the same way as everything else.

Remark

LayerNorm normalizes across *features*, independently for each token; batch normalization (common in convolutional networks) normalizes across the *batch*, independently for each feature. The latter requires batch statistics that are ill-defined for variable-length sequences and awkward at generation time (where sequences are processed one token at a time); LayerNorm has no such issue, which is why it is the default choice in transformers.

Implementation note

Eq. (5.3) is `nn.LayerNorm(n_embd)`, used as a primitive on the same footing as `nn.Linear`; its `weight` and `bias` are γ and β.

5.3 The feed-forward network

Attention is the only sub-layer that lets positions exchange information; by itself it is a (learned, content-dependent) linear recombination of value vectors, with no nonlinearity of its own beyond softmax's weighting. The **feed-forward network** (also called the MLP block) restores a genuine per-position nonlinear transformation, applied identically and independently to every position. It is the map $\mathcal{F} : \mathbb{R}^d \to \mathbb{R}^d$,

$$\mathcal{F}(h) = W_2\,\phi(W_1 h + b_1) + b_2, \qquad W_1 \in \mathbb{R}^{4d\times d},\ W_2 \in \mathbb{R}^{d\times 4d}, \tag{5.6}$$

with $\phi : \mathbb{R} \to \mathbb{R}$ a nonlinearity, applied to each component of its argument separately. As for $\mathcal{N}$, $\mathcal{F}(H)$ means $\mathcal{F}$ applied to each row of H. The hidden dimension $4d$ (a $4\times$ expansion) is the convention introduced with the original Transformer and kept by GPT-2; it is not derived from anything more fundamental. Two common choices of ϕ are:

$$\mathrm{ReLU}(x) = \max(0, x), \tag{5.7}$$

$$\mathrm{GELU}(x) = x\,\Phi(x), \quad \Phi(x) = \tfrac{1}{2}(1 + \mathrm{erf}(x/\sqrt{2})), \tag{5.8}$$

with Φ the standard normal CDF: GELU can be read as ReLU "smoothed" by weighting x by the probability that a standard normal variable is below x, rather than a hard cutoff at 0. GPT-2 and most modern transformers use GELU; the difference from ReLU is a minor training-quality detail rather than a conceptual one.

Equation (5.6) is nothing but the textbook multilayer perceptron with one hidden layer: an affine map, a component-wise nonlinearity, and a second affine map. What is specific to the transformer is how it is used.

- **Position-wise, with shared weights.** The same W_1, b_1, W_2, b_2 are applied to every row h_t of H, and the rows never interact inside $\mathcal{F}$. This amounts to running one small MLP T times, or equivalently to a 1×1 convolution along the sequence. All the exchange of information between positions happens in attention; $\mathcal{F}$ only transforms what each position already holds.

- **Wide in the middle, linear at the output.** Input and output have dimension d, and the hidden layer has $4d$ units: the representation is expanded, passed through ϕ, and contracted back. There is no nonlinearity after W_2, because the output is not used directly but added to the residual stream (Section 5.4).

- **Inside a residual connection, after a LayerNorm.** In the full block, Eq. (5.10), the update is $H' + \mathcal{F}(\mathcal{N}_2(H'))$, so $\mathcal{F}$ learns a *correction* to the current representation rather than a replacement, and its input is always normalized, which keeps the scale of the arguments of ϕ under control.

- **Most of the parameters.** W_1 and W_2 have $8d^2$ parameters, out of about $12d^2$ per block (Eq. (6.3)): roughly two thirds of each block, and of the whole model.

A useful way to read what $\mathcal{F}$ does is as a key–value memory (Geva et al., 2021). This is an interpretation, not a theorem. Each of the $4d$ rows of W_1 acts as a "key" that detects a pattern in h, and the corresponding column of W_2 is a "value" vector, added to the output with a weight given by how strongly the key responded. In this picture, attention gathers information from other positions, while the MLP retrieves associations stored in its weights during training and adds them to the representation of each position.

Implementation note

`gpt_intro/transformer_block.py:MLP` is exactly Eq. (5.6): two `nn.Linear` layers around an activation (`nn.GELU()` or `nn.ReLU()`, selected by the `activation` argument; since the difference between the two choices of Eq. (5.8) is minor, both are offered as an option), with dropout after the second.

5.4 Residual connections and the full block

Stacking many blocks (Chapter 6 uses several) makes the network deep, and deep networks are harder to optimize: a gradient computed at the output has to flow, by the chain rule, through every block on its way back to an early parameter. **Residual connections** address this by having each sub-layer add its output to its input rather than replace it,

$$h \;\leftarrow\; h + f(h), \tag{5.9}$$

where f is the sub-layer, so that it only has to learn a *correction* to add to the running representation. At initialization (or if a sub-layer's output happens to be near zero) Eq. (5.9) reduces to the identity, so the "default" behavior of a block is to pass its input through unchanged — gradients can flow through the addition unimpeded (the Jacobian of $h + f(h)$ with respect to h is $I + J_f$), regardless of how poorly-conditioned J_f itself may be.

Combining Eq. (5.9) with multi-head attention $\mathcal{M}$ (Eq. (5.2)), the feed-forward network $\mathcal{F}$ (Eq. (5.6)), and a LayerNorm $\mathcal{N}$ (Eq. (5.3)) before each sub-layer gives the complete transformer (decoder) block, the map $\mathcal{B} : \mathbb{R}^{T\times d} \to \mathbb{R}^{T\times d}$, $H \mapsto \mathcal{B}(H)$, with

$$\begin{aligned} H' &= H + \mathcal{M}(\mathcal{N}_1(H)), \\ \mathcal{B}(H) &= H' + \mathcal{F}(\mathcal{N}_2(H')). \end{aligned} \tag{5.10}$$

The two LayerNorms $\mathcal{N}_1, \mathcal{N}_2$ have the same form, Eq. (5.3), but each has its own γ, β. Written row by row, Eq. (5.10) updates every h_t in place, first with the output of attention, which reads $h_1, \ldots, h_t$, and then with $\mathcal{F}(\mathcal{N}_2(h_t))$, which reads h_t alone. The block is therefore a map from sequences to sequences, and $\mathcal{M}$ is the only part of it that lets positions communicate. Applying LayerNorm *before* each sub-layer (**pre-norm**), rather than after the residual addition as in the original 2017 Transformer (**post-norm**), keeps the residual stream itself unnormalized end to end and is the convention adopted from GPT-2 onward because it makes deep stacks of blocks noticeably easier to train; it is the version implemented here. Figure 5.1 shows the data flow of Eq. (5.10).

Note how this changes the role of the context vectors of Chapter 4. In Eq. (4.5) the attention output c_t was mapped to logits directly. Inside a block, the multi-head output at position t (built from the c_t of every head) is instead a *correction* added to h_t, which then goes through the feed-forward network and, in a stack of blocks, through all subsequent blocks. The linear map to logits, Eq. (2.2), moves to the very end of the stack (Section 6.2), where it is applied to the final h_t.

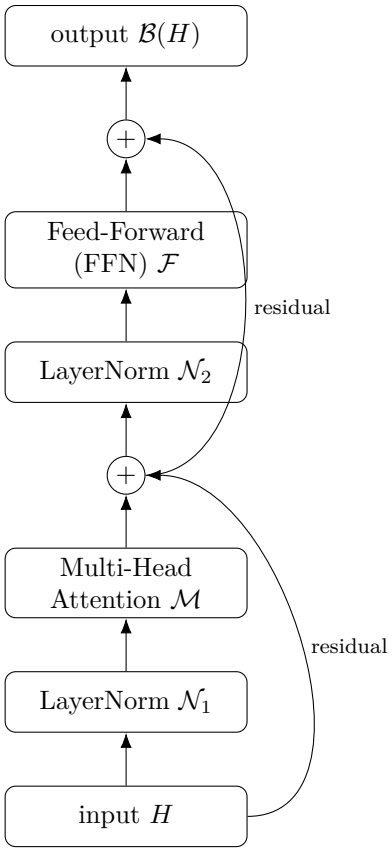


Figure 5.1: One transformer block, pre-norm. Each of the two sub-layers (attention, feed-forward) reads a LayerNorm'd copy of the running representation and its output is *added* to the unnormalized residual stream, following Eq. (5.10).

Implementation note

`gpt_intro/transformer_block.py:Block` implements Eq. (5.10) in two lines: `x = x + self.attn(self.ln1(x))` and `x = x + self.mlp(self.ln2(x))`, using `MultiHeadAttentionFast` from the previous section for `self.attn`.

Chapter 6

Assembling GPT

Everything needed is now in hand: an embedding table (Section 2.4), self-attention (Chapter 4), and the transformer block (Chapter 5). This chapter assembles them into the full architecture and accounts for its parameters.

It is worth looking back at how we got here. Each step of the previous chapters inserted a new intermediate representation between the tokens and the logits, while the last step of the chain — a linear map to logits, Eq. (2.2) — stayed the same; what changed is the vector it is applied to (Table 6.1).

Model	Chain from tokens to logits	New intermediate
Bigram (Ch. 3)	$x_{t-1} \to z_t = Z_{x_{t-1}}$	—
Bag of words (Sec. 3.6)	$x_{\le t-1} \to e_{x_s} \to c_{t-1} \to z_t = W c_{t-1}$	e_{x_s}, c_{t-1}
Attention (Ch. 4)	$x_{\le t} \to e_{x_s} \to q_s, k_s, v_s \to c_t \to z_{t+1} = W c_t$	q_s, k_s, v_s
Block (Ch. 5)	$h_t = e_{x_t} \to \mathcal{M}$, $\mathcal{F}$ updates of $h_t \to z_{t+1} = W h_t$	h_t (residual stream)
GPT (this chapter)	$h_t^{(0)} = e_{x_t} + p_t \to h_t^{(1)} \to \cdots \to h_t^{(L)} \to z_{t+1}$	p_t, $h_t^{(\ell)}$

Table 6.1: The ladder of intermediate representations. Representations are indexed by the last token they have read, logits by the token they predict (Section 2.1).

6.1 Why position has to be added by hand

Look back at Eq. (4.6): attention computes, for each row (query) t, a weighted sum of value rows, where the weights come from dot products between rows of Q and K. Nothing in that computation refers to t or s as *positions* — only to the content of the rows. If the rows of H (and hence of Q, K, V) were permuted, the output rows would be permuted by exactly the same permutation, with the same values in them: attention is **permutation-equivariant**. Except for the causal mask, which constrains *which* pairs may interact (Section 4.3), attention has no built-in notion of order at all — unlike a recurrent network, which processes tokens one at a time in sequence and so has order "for free".

Since order plainly matters for language, position must be injected explicitly, as part of the input representation itself. The simplest way — and the one used in GPT-2, which we follow here — is a second, learned embedding table indexed not by token identity but by *position*: $E_{\text{pos}} \in \mathbb{R}^{L_{\max} \times d}$, where $L_{\max}$ is the maximum sequence length the model will ever be run on (called the **context**

length). The input to the first transformer block is the sum of the two embeddings,

$$h_t^{(0)} = e_{x_t} + p_t, \tag{6.1}$$

where e_{x_t} is the row of the token embedding table $E_{\text{tok}} \in \mathbb{R}^{V \times d}$ (the E of Section 2.4) selected by the token id x_t, and p_t is the row of E_{pos} selected by the position t. This $h_t^{(0)}$ is the initial value of the h_t of Chapter 5, as seen by the first block; more generally we write $h_t^{(\ell)}$ for the output of block ℓ at position t. Adding p_t means that the same token appearing at two different positions arrives at the first block with two different (position-shifted) representations, and the model is free to learn how to use that shift.

Remark

Learned positional embeddings are simple but fixed in advance: a model trained with context length $L_{\max}$ has no p_t for $t > L_{\max}$ and so cannot be run, as is, on longer sequences. The original Transformer instead used fixed sinusoidal functions of position (which can in principle be evaluated at any t), and more recent architectures use *relative* position schemes such as RoPE that encode the distance $t - s$ directly into the attention computation rather than adding anything to $h_t^{(0)}$. We use learned absolute positional embeddings because they are what GPT-2 uses and because they require no additional machinery beyond an embedding table; extending context length beyond $L_{\max}$ is a real limitation worth returning to once we scale up.

6.2 The full forward pass

Given input token ids $x_1, \ldots, x_T$ (with $T \le L_{\max}$), the model:

1. embeds tokens and positions and adds them, Eq. (6.1);
2. passes the result through L transformer blocks in sequence, Eq. (5.10): writing $H^{(\ell)} \in \mathbb{R}^{T \times d}$ for the matrix with rows $h_1^{(\ell)}, \ldots, h_T^{(\ell)}$, block ℓ computes $H^{(\ell)} = \mathcal{B}_\ell(H^{(\ell-1)})$, where $\mathcal{B}_\ell$ is the map $\mathcal{B}$ of Eq. (5.10) with the parameters of block ℓ;
3. applies one final LayerNorm $\mathcal{N}_f$, with its own γ, β, to the output of the last block, $h_t := \mathcal{N}_f(h_t^{(L)})$;
4. projects to vocabulary logits with one more linear layer, $W_{\text{head}} \in \mathbb{R}^{V \times d}$ (no bias), giving $z_{t+1} = W_{\text{head}} h_t \in \mathbb{R}^V$ at every position t. This is the map from context vector to logits of Eq. (2.2), with $W = W_{\text{head}}$ and the output h_t of the transformer stack playing the role of the context vector.

In terms of the maps of Chapters 4 and 5, steps 2–4 are a single composition,

$$\begin{bmatrix} z_2^\top \\ \vdots \\ z_{T+1}^\top \end{bmatrix} = \mathcal{N}_f(\mathcal{B}_L \circ \cdots \circ \mathcal{B}_1(H^{(0)}))\, W_{\text{head}}^\top \in \mathbb{R}^{T \times V}, \tag{6.2}$$

where $H^{(0)}$ has rows $h_1^{(0)}, \ldots, h_T^{(0)}$, Eq. (6.1). Since h_t has read $x_1, \ldots, x_t$, the logits z_{t+1} computed from it score the candidates for the *next* token x_{t+1}. A single forward pass over $x_1, \ldots, x_T$ therefore produces T predictions at once: $z_2, \ldots, z_T$, compared against the actual tokens $x_2, \ldots, x_T$ during training, and z_{T+1}, a genuine prediction beyond the end of the sequence, used during generation. Steps 1–4 define $p(x_{t+1} \mid x_{\le t})$ exactly as in Chapter 1: z_{t+1} is fed to Eq. (2.5) during training and to Eq. (2.3) (optionally reshaped by temperature, see Section 7.3) during generation. Note that the causal mask inside every block (Section 4.3) is what guarantees that z_{t+1}, and hence $p(x_{t+1} \mid \cdot)$,

depends only on $x_1, \ldots, x_t$ and never on the token x_{t+1} it predicts nor on any later one, even though the whole sequence is passed through the model at once.

Implementation note

`gpt_intro/gpt.py:GPT.forward` implements steps 1–4 directly: `self.token_emb(idx) + self.pos_emb(positions)`, a loop over `self.blocks`, `self.ln_f`, then `self.head`. The context length $L_{\max}$ is `GPTConfig.block_size`, and `self.pos_emb` is an `nn.Embedding(block_size, n_embd)`.

6.3 Weight tying

The token embedding table $E_{\text{tok}} \in \mathbb{R}^{V \times d}$ (step 1) and the output head $W_{\text{head}} \in \mathbb{R}^{V \times d}$ (step 4) have the *same shape* and a related job: one maps a token id to a vector, the other maps a vector back to a score for every token id. GPT-2 (following Press & Wolf, 2017) ties them together, $W_{\text{head}} = E_{\text{tok}}$, a single parameter tensor used both ways. This removes one of the two largest tensors in the whole model (for the character-level vocabulary used here, $V = 65$, this only saves a few thousand parameters, but for a realistic sub-word vocabulary of $V \approx 50{,}000$ it removes tens of millions) and, empirically, tends to improve language-modeling quality rather than merely economize on parameters.

Implementation note

`gpt_intro/gpt.py:GPT.__init__` sets `self.head.weight = self.token_emb.weight` after constructing both layers: this makes `head.weight` and `token_emb.weight` literally the same tensor object, so a gradient step updates both "uses" simultaneously and `model.num_params()` does not double-count it.

6.4 Parameter count

It is useful to be able to predict a GPT's parameter count from its hyperparameters alone, both to reason about model size before training and to sanity-check an implementation. Per block, the dominant terms are the attention projections (W_Q, W_K, W_V, W_O, each $d \times d$, for $4d^2$ total) and the feed-forward network ($W_1 \in \mathbb{R}^{4d \times d}$ and $W_2 \in \mathbb{R}^{d \times 4d}$, for $8d^2$ total):

$$N_{\text{block}} \approx 4d^2 + 8d^2 = 12\,d^2 \tag{6.3}$$

(LayerNorm's γ, β and the linear biases add only $O(d)$ terms per block and are dropped from this estimate). With L blocks, plus the (tied) token embedding Vd and the positional embedding $L_{\max}d$,

$$N_{\text{total}} \approx 12\,L\,d^2 + Vd + L_{\max}d. \tag{6.4}$$

This is the same rule of thumb used to describe the size of GPT-2/GPT-3 scale models (where the embedding terms are negligible next to $12Ld^2$). For the demo configuration used in our experiments ($L = 4$, $d = 128$, $V = 65$, $L_{\max} = 128$), Eq. (6.4) gives $12 \cdot 4 \cdot 128^2 + 65 \cdot 128 + 128 \cdot 128 \approx 786{,}432 + 8{,}320 + 16{,}384 = 811{,}136$, within 1% of the exact count of 816,512 (the difference is exactly the biases and LayerNorm parameters dropped from Eq. (6.3)).

Implementation note

The demo configuration is the one in `scripts/train_gpt.py`, and the exact count 816,512 is the value reported by `model.num_params()`.

Chapter 7

Training and generation

7.1 Teacher forcing and batching

A GPT is trained on ordinary text, with no special labels: the target for position t is simply the token that actually occurs at position $t+1$ in the corpus. Given a chunk of T consecutive tokens starting at some offset i in the corpus, $x_i, \ldots, x_{i+T-1}$, the input sequence and target sequence are the *same* chunk shifted by one:

$$\text{input} = (x_i, \ldots, x_{i+T-1}), \qquad \text{target} = (x_{i+1}, \ldots, x_{i+T}). \tag{7.1}$$

Because of the causal mask (Section 4.3), the model's prediction at position t depends only on inputs up to t, so computing all T next-token predictions for the whole chunk in a single forward pass is valid: this is what makes training on sequences parallelizable across positions, unlike a recurrent model. Feeding the model the *true* tokens as input at every position (rather than its own, possibly wrong, predictions from previous steps) is called **teacher forcing**; it is what Eq. (7.1) does, and it is standard for training autoregressive models, even though at generation time (Section 7.3) the model necessarily does condition on its own previous outputs.

A training batch is simply B such chunks, sampled at random starting offsets i from the corpus, stacked into a tensor of shape (B, T); the loss (Eq. (1.2)) is averaged over both the batch and the T positions, giving one scalar per optimization step.

Implementation note

`gpt_intro/data.py:Dataset.get_batch` implements Eq. (7.1): it draws `batch_size` random starting indices and slices out `x = ids[i:i+block_size]`, `y = ids[i+1:i+block_size+1]` for each, with `block_size` $= T$. A held-out `val` split (the last `val_fraction` of the corpus, never sampled for training batches) is used to monitor generalization, following the train/validation split already used in earlier PyTorch guides of this course.

7.2 Optimization: what autograd and AdamW are doing

None of the equations in these notes describe how gradients are computed — deliberately. Gradients are obtained by reverse-mode automatic differentiation, which walks the computation graph built by every tensor operation in the forward pass (matrix multiplications, softmax, LayerNorm, embedding lookups, and so on) backward, applying the chain rule mechanically at each node to accumulate $\partial \mathcal{L}/\partial \theta$ for every parameter θ in the model. This is exactly the autograd engine used for every other

model in this course; nothing about attention or the transformer block requires it to be treated specially, which is precisely why building these architectures out of ordinary linear layers, embedding lookups and tensor operations (as in Chapters 3–6) is enough to make them trainable.

Given a gradient, **AdamW** updates each parameter using running estimates of the first and second moment of its own gradient history (an exponential moving average of the gradient, and of the squared gradient), rather than the raw gradient itself as plain SGD would. Concretely, writing g for the gradient of $\mathcal{L}$ with respect to a single parameter at step n,

$$m_n = \beta_1 m_{n-1} + (1-\beta_1) g, \qquad v_n = \beta_2 v_{n-1} + (1-\beta_2) g^2, \tag{7.2}$$

$$\theta \leftarrow \theta - \eta \left(\frac{\hat{m}_n}{\sqrt{\hat{v}_n} + \epsilon} + \lambda \theta \right), \tag{7.3}$$

with $\hat{m}_n, \hat{v}_n$ bias-corrected versions of m_n, v_n and λ a weight-decay coefficient applied directly to θ (the "decoupled" part of AdamW, as opposed to folding weight decay into g as in plain Adam). The practical consequence of Eq. (7.3) is that every parameter gets its own effective learning rate, adapted to the typical magnitude of its own gradient — useful here because the model has parameters of very different character (a 65×128 embedding row updated only when its token appears in a batch, versus a 128×128 attention projection updated on every batch), which a single shared learning rate would treat far less gracefully.

Remark

Real GPT training uses a learning-rate *schedule* on top of Eq. (7.3), typically a brief linear warmup (to avoid large, destabilizing updates while m_n, v_n are still poorly estimated at the very start of training) followed by cosine decay. Our demo uses a single fixed learning rate for simplicity, which is adequate at this scale (a few million parameters, a few thousand steps) but becomes an important knob once training runs longer and at the larger scale of the planned cluster phase (Section 7.4).

Implementation note

`gpt_intro/train.py:train` is the shared training loop: sample a batch, compute the loss, `optimizer.zero_grad()`, `loss.backward()`, `optimizer.step()`, repeated for `max_iters` steps, with periodic evaluation (`estimate_loss`) of the average loss on both splits under `torch.no_grad()`. Here `loss.backward()` is the reverse-mode automatic differentiation of Section 7.2, which works unchanged because the model is built from ordinary `nn.Linear`, `nn.Embedding` and tensor operations, and the optimizer is `torch.optim.AdamW`, implementing Eq. (7.3). The demo `scripts/train_gpt.py` uses the fixed learning rate $\eta = 3 \times 10^{-4}$, without the schedule discussed in the remark above.

7.3 Generation

Generation applies $p(x_t \mid x_{<t})$ (Section 6.2) repeatedly: sample $\hat{x}_t$ from the model's predicted distribution over the next token, append it to the sequence, and feed the extended sequence back in for the next step (truncating to the last $L_{\max}$ tokens if the sequence has grown longer than the model's context length). Two adjustments to the raw distribution are standard:

- **Temperature** τ rescales the logits before softmax, $p_\tau(x_t) = \sigma_{x_t}(z_t/\tau)$: $\tau < 1$ sharpens the distribution toward the model's most likely tokens (more repetitive, less diverse text), $\tau > 1$ flattens it (more diverse, more likely to be incoherent); $\tau = 1$ recovers the model's own distribution unchanged.

- **Top-k filtering** zeroes out every logit outside the k largest before softmax, so that only the k most likely next tokens can ever be sampled, no matter how much probability mass the tail of the distribution would otherwise carry.

Implementation note

`gpt_intro/gpt.py:GPT.generate` implements exactly this loop, including both adjustments (`temperature`, `top_k`); the same pattern (without temperature or top-k) is used for the bigram models in `gpt_intro/bigram.py` for comparison.

7.4 Where this is going

The model assembled in these notes – a handful of transformer blocks, trained on a megabyte of text, on a laptop CPU – is a genuine (if tiny) GPT: the same equations, unchanged, describe GPT-2 and scale, with more blocks, wider layers, a sub-word tokenizer, far more data, and many GPUs, to far more capable models. The next phase of this course moves along exactly that axis rather than introducing new architecture: distributed data loading, mixed-precision training, multi-GPU parallelism, and running training jobs on CCAD's A30 cluster nodes. None of it changes a single equation in this chapter.